

Laboratório de Concorrência · Aula 10

Guilherme Yukio Yamaguchi · RA 24062624-2 · Turma ES6S-A

Gerado em 09/10/2026, 21:05:58 · tempo registrado no cronômetro: 0:00

Preparação: Concorrência em cinco minutos

0.1 Com suas palavras: qual é a diferença entre sincronização de cooperação e de competição? Dê um exemplo do dia a dia (fora da computação) para cada uma.

Cooperação: ocorre quando uma tarefa precisa esperar outra tarefa ser executada primeiro. Ex: Fila em mercado

Competição: ocorre quando duas tarefas ou mais compartilham um mesmo recurso para serem executadas. Ex: Duas pessoas tentando utilizar uma impressora ao mesmo tempo

Prática 1: A condição de corrida

Previsão: Duas threads somam 1.000.000 cada. Que valor você espera que o programa imprima?

1,000.000

1ª execução (n = 1.000.000)

1520945

2ª execução (n = 1.000.000)

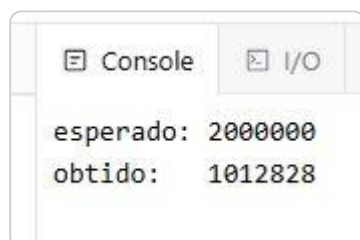
1012828

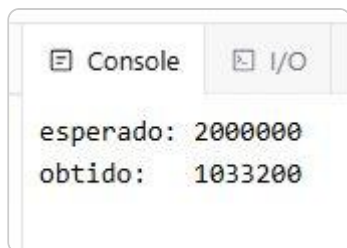
3ª execução (n = 1.000.000)

1033200

Saída do programa

não respondido





1.1 Os valores bateram com o esperado? Foram iguais nas três execuções? Por que mudam de uma execução para outra?

Os valores foram diferentes nas três execuções, pois existe uma condição de corrida, onde as duas threads somam no contador sem saber qual número cada uma possui.

1.2 Explique, passo a passo, como uma atualização se perde. Use os três passos (ler, somar, gravar) e as duas threads.

Uma atualização se perde quando ela lê o valor do contador e guarda em seu registrador local, soma um novo valor para o mesmo registrador e grava no processador esse novo valor somado, apagando o valor antigo que estava lá e deixando a sequência fora de ordem, pois a outra thread que compartilha da mesma variável está com o valor antigo.

1.3 Com $n = 1000$, o que aconteceu? Por que isso torna esse tipo de erro perigoso em sistemas reais? Pense em como o programa se comportaria nos testes e em produção.

Com $n = 1000$, não houve condição de corrida. A condição de corrida pode ser perigosa quando se precisa ter previsibilidade dos resultados e consistência de dados, principalmente em testes e em produção, pois erros precisam ser replicáveis para serem solucionados.

Prática 2: Todas as respostas possíveis (Figura 13.1)

Ordens testadas no simulador

ABABAB → 6

AAABBB → 8

BBBAAA → 7

AABBAB → 6

BBAABA → 4

AABBBA → 4

Saída do programa

não respondido

```

Console I/O AI Agent
AAABBB -> 8
BBBAAA -> 7
ABABAB -> 6
Ordens possíveis: 20
TOTAL = 4: 9 ordens, por exemplo AABBBB
TOTAL = 6: 9 ordens, por exemplo AABABB
TOTAL = 7: 1 ordens, por exemplo BBBAAA
TOTAL = 8: 1 ordens, por exemplo AAABBB

```

2.1 Quantas ordens são possíveis e quantos resultados diferentes aparecem? Quantas ordens dão o resultado "correto" (8)?

São possíveis 20 ordens com 4 resultados diferentes. Apenas uma ordem possui o resultado igual a 8(AAABBB)

2.2 Escolha uma ordem que dá 4 e explique passo a passo por que o resultado é 4. O que aconteceu com a operação de B?

AABBBB possui resultado 4 pois ao executar todas as duas tarefas de A primeiro e todas as tarefas de B depois, ainda existe uma atividade de A para realizar, que apaga o resultado da B que estava em seu registrador

Prática 3: Corrigindo a corrida

Previsão: As três versões fazem a mesma conta. Qual você acha que será a mais rápida? E alguma será incorreta?

Ao meu ver, a operação atômica parece ser mais rápida, e nenhuma outra estará incorreta

Saída do programa

não respondido

```

Console I/O AI Agent
esperado:          2000000
sem sincronizacao: 1399809 em 2 ms
synchronized:      2000000 em 41 ms
AtomicInteger:     2000000 em 57 ms

```

```

Console I/O AI Agent
esperado:          2000000
sem sincronizacao: 1024987 em 2 ms
synchronized:      2000000 em 119 ms
AtomicInteger:     2000000 em 71 ms

```

3.1 Quais versões deram o valor correto? Compare os tempos: por que a versão correta costuma ser mais lenta que a versão sem sincronização?

A `synchronized` e a `AtomicInteger` deram os valores corretos. A versão correta é mais lenta pois ela precisa garantir que as threads não fiquem fora de sincronização

3.2 Uma transferência bancária tira dinheiro de uma conta e coloca em outra. Um `AtomicInteger` em cada saldo resolve? Por quê? O que seria preciso?

Um `AtomicInteger` não seria o suficiente, pois seria necessário uma operação atômica global, onde as duas operações para sacar dinheiro de uma conta e transferir em outra ocorreriam de forma sincronizada.

Prática 4: Impasse (deadlock)

Previsão: Leia o código: A pega X e depois Y; B pega Y e depois X. As mensagens "A terminou" e "B terminou" vão aparecer?

Elas vão aparecer caso não existam mais possibilidades de continuar o programa

Saídas (antes e depois da correção)

não respondido

```
Console I/O AI Agent
IMPASSE: 2 threads presas
```

```
Console I/O AI Agent
A terminou
B terminou
sem impasse
```

4.1 Por que as duas threads ficaram presas? Descreva o que cada uma tinha e o que cada uma esperava.

As duas threads ficam presas pois ambas possuem um recurso que a outra thread precisa, e como elas não liberam esses recursos até terminarem sua atividade, acabam em impasse

4.2 Por que pegar os recursos sempre na mesma ordem elimina o impasse? Dê um exemplo de sistema real em que isso poderia acontecer.

Pois cada thread poderá utilizar seu recurso que já possui, eliminando a situação de uma thread precisar de um recurso de outra thread para continuar. Exemplo: Banco de dados de transações, onde duas transações precisam atualizar os mesmos dois clientes.

Prática 5: Mensagens em vez de memória compartilhada

Previsão: São 6 tarefas de 100 ms divididas entre 3 trabalhadores. Quanto tempo você espera que leve no total? E com 1 trabalhador? E com 6?

Espera-se que leve 200 ms com 3 trabalhadores, 600 ms com 1 trabalhador e 100 ms com 6 trabalhadores

Tempo com 1 trabalhador

600

Tempo com 3 trabalhadores

200

Tempo com 6 trabalhadores

100

Saída do programa*não respondido*

```
Console I/O AI Agent
5 trabalhador 2 fez a tarefa 3
  trabalhador 3 fez a tarefa 1
  trabalhador 1 fez a tarefa 2
  trabalhador 2 fez a tarefa 4
  trabalhador 1 fez a tarefa 6
  trabalhador 3 fez a tarefa 5
  3 trabalhadores, 6 tarefas de 100 ms: levou 200ms
```

```
Console I/O AI Agent
5 trabalhador 1 fez a tarefa 1
  trabalhador 1 fez a tarefa 2
  trabalhador 1 fez a tarefa 3
  trabalhador 1 fez a tarefa 4
  trabalhador 1 fez a tarefa 5
  trabalhador 1 fez a tarefa 6
  1 trabalhadores, 6 tarefas de 100 ms: levou 600ms
```

```
Console I/O AI Agent
6 trabalhador 2 fez a tarefa 3
  trabalhador 5 fez a tarefa 6
  trabalhador 6 fez a tarefa 1
  trabalhador 1 fez a tarefa 2
  trabalhador 3 fez a tarefa 4
  trabalhador 4 fez a tarefa 5
  6 trabalhadores, 6 tarefas de 100 ms: levou 100ms
```

5.1 Explique os três tempos. Por que este programa não precisa de synchronized nem de trava?

Pois o programa utiliza canais internos para se comunicar e sincronizar, onde dados passam de uma goroutine para outra, evitando que haja múltiplos acessos na mesma variável ao mesmo tempo, evitando situação de corrida

Saídas (antes e depois da troca)*não respondido*

```
Console I/O AI Agent

fatal error: all goroutines are asleep - deadlock!

goroutine 1 [chan send]:
main.main()
    /box/main.go:9 +0x38
exit status 2
```

```
Console I/O AI Agent

1
```

5.2 Por que o primeiro programa trava? Por que o canal com espaço para 1 mensagem resolve? (Dica: num canal sem espaço, quem envia espera alguém receber.)

Pois o Go trabalha com canais para envio de mensagens e, no primeiro programa não havia espaço para guardar mensagem, diferente do segundo, onde o buffer consegue armazenar uma mensagem e realizar o envio de informação

Prática 6: O compilador como guarda

Previsão: Este programa faz uma thread somar 1 a uma variável de main. O que você acha que acontece?

O programa não terminara pois a linguagem não permite

Saídas (o erro e o resultado com move)

não respondido

```
Console I/O AI Agent Exit 1

error[E0373]: closure may outlive the current function, but it
borrows `contador`, which is owned by the current function
--> HelloWorld.rs:7:27
   |
7  |     let t = thread::spawn(|| {
   |                               ^^ may outlive borrowed value
`contador`
```

```
Output:

0
```

Saída do programa

não respondido



6.1 Com move, o programa compila mas imprime 0. Por quê? (Dica: move faz a thread levar uma cópia do número.)

Pois o move transfere a propriedade ou copia os valores capturados para a closure

6.2 Compare com a Prática 1: em que momento o erro foi descoberto em Java e em Rust? Qual é a vantagem e qual é o custo para quem programa?

Em Java, o erro é descoberto após a compilação, enquanto no Rust avisa antes quando existe algum erro. Sua maior vantagem é detecção de erros e problemas antes de execução, com seu custo sendo mais tempo sendo gasto para satisfazer as regras do compilador, entendendo ownership, empréstimos e tempos de vida

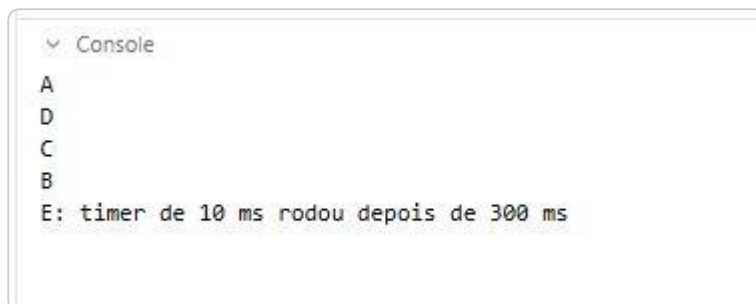
Prática 7: Uma thread só: o laço de eventos

Previsão: Em que ordem você acha que as letras A, B, C e D serão impressas?

A, B, D, C, E

Saída do programa

não respondido



7.1 Em que ordem as letras saíram? Por que B, com tempo 0, saiu depois de D e de C?

A, D, C, B, E. Pois a linguagem deixa os callbacks para serem executados depois dos console.logs, com as microtasks tendo prioridade

7.2 O timer de 10 ms (linha E) rodou bem depois de 10 ms. Por quê? O que isso significa para um servidor Node.js que atende muitos usuários e faz um cálculo pesado?

Pois o setTimeout garante apenas um tempo mínimo de espera, com o ocupado travando a operação por 300 ms, fazendo com que o E seja imprimido por ultimo

Fechamento: Quem impede a corrida?

Java sem sincronização

Ninguém: há corrida

Java com synchronized

O programador, com sincronização

Go com canais

O projeto: não há memória compartilhada

Rust com Arc e Mutex

O compilador

JavaScript

Só existe uma thread

8.1 Um site de venda de ingressos tem 1 lugar restante, e duas pessoas clicam em Comprar ao mesmo tempo.

Qual problema desta atividade pode acontecer? Como você o evitaria?

O problema apresentado é uma condição de corrida, que pode ser evitada com a tela de compra de ingresso sendo travada enquanto uma compra está sendo processada, impedindo que outra transação o venda simultaneamente

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

Condições de compartilhamento de variáveis e dados requer muita atenção do programador